

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA0401

COURSE OUTCOME COVERED

CO5: Analyze the components and functionalities of Linux operating systems and virtualization environments to enhance their operational efficiency

Assessment Tool Weightage: 35%

QUESTIONS: 5

Total Marks:25

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I G. Yashmita (192511032) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G. Yashmita

Annexure A

PSEUDOCODE WRITING TASK

S.No.	Questions	Mark
1	A system is at risk of disk overflow due to continuously increasing log files. Analyze the situation and develop a file system space management algorithm, using pseudocode, to monitor disk usage and automatically delete or rotate log files whenever disk utilization exceeds 80%. Pseudocode:BEGIN SET threshold = 80% usage = GET_DISK_USAGE() IF usage > threshold THEN IDENTIFY log_files SORTED BY size DESC FOR each file IN log_files DO DELETE or COMPRESS file usage = GET_DISK_USAGE() IF usage <= threshold THEN BREAK ENDIF	5

	ENDFOR ENDIF END	
2	Improve CPU efficiency in a system by designing a priority-based scheduling algorithm. Develop and present the corresponding pseudocode to demonstrate how processes are selected and executed based on their priority levels. <pre> Pseudocode:BEGIN INPUT process_list SORT process_list BY priority (highest first) WHILE process_list NOT EMPTY DO SELECT process WITH highest priority ALLOCATE CPU time EXECUTE process IF process NOT completed THEN REDUCE priority slightly ADD back to process_list ENDIF ENDWHILE END </pre>	5
3	Efficiently manage and distribute RAM resources among multiple virtual machines in a system to ensure balanced performance and optimal utilization. <pre> Pseudocode:BEGIN total_RAM = 16GB host_reserved = 4GB available_RAM = total_RAM - host_reserved INPUT number_of_VMs min_RAM = 2GB FOR i = 1 TO number_of_VMs DO IF available_RAM >= min_RAM THEN ALLOCATE min_RAM to VM[i] available_RAM = available_RAM - min_RAM ELSE PRINT "Insufficient memory" ENDIF ENDIF </pre>	5
4	Select the most appropriate hypervisor for a given system environment by analyzing the specified requirements and constraints, and justify your decision to ensure optimal performance and efficient resource utilization. <pre> Pseudocode:BEGIN INPUT performance_required, budget, hardware_access IF performance_required = HIGH AND hardware_access = TRUE THEN SELECT Type_1_Hypervisor ELSE IF budget = LOW THEN SELECT Type_2_Hypervisor ELSE SELECT Type_1_Hypervisor ENDIF PRINT selected_hypervisor END </pre>	5
5	Manage and regulate CPU and memory usage of applications in a system by analyzing the scenario and implementing suitable resource-limiting mechanisms to ensure balanced and efficient system performance. <pre> Pseudocode:BEGIN INPUT application_list </pre>	5

	FOR each app IN application_list DO CREATE cgroup for app SET CPU limit SET memory limit ASSIGN process to cgroup ENDFOR END	
--	--	--

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. File System Space Management Algorithm:

Problem Analysis:

The disk may become full because log files continuously increase in size. To prevent disk overflow, the system should continuously monitor disk utilization. When usage exceeds 80%, large log files should be compressed, rotated, or deleted until disk usage returns to 80% or below.

Pseudocode:

```
BEGIN
  SET threshold = 80%
  WHILE system is running DO
    usage = GET_DISK_USAGE()
    IF usage > threshold THEN
      PRINT "Warning: Disk usage exceeds 80%"
      log_files = GET_LOG_FILES()
      SORT log_files BY size DESCENDING
      FOR each file IN log_files DO
        IF file is old THEN
          COMPRESS file
        ELSE
          ROTATE file
        ENDIF
        usage = GET_DISK_USAGE()
        IF usage <= threshold THEN
          BREAK
        ENDIF
      ENDFOR
    ENDIF
    IF usage > threshold THEN
      DELETE oldest unnecessary log files
    ENDIF
  ENDWHILE
END
```

Explanation :

The algorithm continuously checks disk usage. If usage exceeds 80%, log files are sorted according to size and old logs are compressed or rotated. If the disk is still above the threshold, unnecessary old logs are deleted.

CODE :

```
#include <stdio.h>
int main() {
  float usage;
  int logFiles, i;
  float fileSize[10];
  printf("Enter current disk usage percentage: ");
  scanf("%f", &usage);
  printf("Enter number of log files: ");
```

```

scanf("%d", &logFiles);
printf("Enter size of each log file in GB:\n");
for (i = 0; i < logFiles; i++) {
printf("Log file %d: ", i + 1);
scanf("%f", &fileSize[i]);
}
printf("\nInitial Disk Usage: %.2f%%\n", usage);
if (usage > 80) {
printf("Disk usage exceeds 80%%.\n");
printf("Starting log management...\n");
for (i = 0; i < logFiles; i++) {
if (usage > 80) {

printf("Compressing/rotating Log file %d (%.2f GB)\n",
i + 1, fileSize[i]);
usage = usage - (fileSize[i] * 2);
} else {
break;
}
}
if (usage > 80)
printf("Old log files need to be deleted.\n");
} else {
printf("Disk usage is within safe limit.\n");
}
printf("Final Disk Usage: %.2f%%\n", usage);
if (usage <= 80)
printf("Constraint satisfied: Disk usage <= 80%%\n");
else
printf("Constraint not satisfied.\n");
return 0;
}

```

OUTPUT :

```

Output

Enter current disk usage percentage: 85
Enter number of log files: 3
Enter size of each log file in GB:
Log file 1: 1
Log file 2: 2
Log file 3: 1

Initial Disk Usage: 85.00%
Disk usage exceeds 80%.
Starting log management...
Compressing/rotating Log file 1 (1.00 GB)
Compressing/rotating Log file 2 (2.00 GB)
Final Disk Usage: 79.00%
Constraint satisfied: Disk usage <= 80%

*** Code Execution Successful ***

```

2. Priority-Based CPU Scheduling Algorithm :

Problem Analysis:

When multiple processes compete for CPU time, processes with higher priority should be executed first. To avoid starvation of lower-priority processes, the priority of a process that repeatedly waits can be increased.

Pseudocode:

```
BEGIN
  INPUT process_list
  FOR each process IN process_list DO
    INPUT process.priority
    INPUT process.burst_time
  ENDFOR
  SORT process_list BY priority
  HIGHEST priority first
  WHILE process_list is NOT EMPTY DO
    SELECT process with highest priority
    ALLOCATE CPU to selected process
    EXECUTE process for its time quantum
    IF process is completed THEN
      REMOVE process from process_list
    ELSE
      REDUCE its priority slightly
      INCREASE waiting process priority
      ADD process back to process_list
    ENDIF
    SORT process_list BY priority
  ENDWHILE

  PRINT "All processes completed"
END
```

Explanation :

The scheduler selects the process with the highest priority and gives it CPU time. If the process is not completed, it is placed back into the ready queue. Priority adjustment can prevent starvation and improve CPU efficiency.

CODE :

```
#include <stdio.h>

struct Process {
  int pid;
  int priority;
  int burst;
};

int main() {
  int n, i, j;
```

```

struct Process p[10], temp;
printf("Enter number of processes: ");
scanf("%d", &n);
for (i = 0; i < n; i++) {
printf("Enter Process ID, Priority and Burst Time for P%d: ", i + 1);
scanf("%d %d %d", &p[i].pid, &p[i].priority, &p[i].burst);
}
for (i = 0; i < n - 1; i++) {
for (j = i + 1; j < n; j++) {
if (p[i].priority > p[j].priority) {
temp = p[i];
p[i] = p[j];
p[j] = temp;
}
}
}

printf("\nPriority Scheduling Order:\n");
for (i = 0; i < n; i++) {
printf("P%d -> Priority = %d, Burst Time = %d\n", p[i].pid, p[i].priority, p[i].burst);
}
printf("\nCPU Execution:\n");
for (i = 0; i < n; i++) {
printf("Executing P%d\n", p[i].pid);
}
printf("\nAll processes completed.\n");
return 0;
}

```

OUTPUT :

```

Output
Enter number of processes: 4
Enter Process ID, Priority and Burst Time for P1: 1 3 4
Enter Process ID, Priority and Burst Time for P2: 5 3 5
Enter Process ID, Priority and Burst Time for P3: 3 4 7
Enter Process ID, Priority and Burst Time for P4: 2 6 4

Priority Scheduling Order:
P1 -> Priority = 3, Burst Time = 4
P5 -> Priority = 3, Burst Time = 5
P3 -> Priority = 4, Burst Time = 7
P2 -> Priority = 6, Burst Time = 4

CPU Execution:
Executing P1
Executing P5
Executing P3
Executing P2

All processes completed.

```

3. RAM Resource Management Among Virtual Machines :

Problem Analysis:

The system has limited physical RAM. The host operating system must first receive its reserved memory. The remaining memory can then be distributed among VMs. Each VM should receive its minimum required memory before additional memory is assigned.

Pseudocode :

```
BEGIN
  total_RAM = 16 GB
  host_reserved = 4 GB
  available_RAM = total_RAM - host_reserved
  INPUT number_of_VMs
  min_RAM = 2 GB
  FOR i = 1 TO number_of_VMs DO
    IF available_RAM >= min_RAM THEN
      ALLOCATE min_RAM TO VM[i]
      available_RAM = available_RAM - min_RAM
    ELSE
      PRINT "Insufficient memory for VM[i]" STOP allocation
    ENDIF
  ENDFOR
  PRINT "RAM allocation completed" PRINT "Remaining RAM =", available_RAM
  IF available_RAM > 0 THEN

    DISTRIBUTE remaining RAM based on VM workload
  ENDIF
END
```

Explanation :

First, 4 GB is reserved for the host. From the remaining 12 GB, each VM receives at least 2 GB. Any remaining memory can be distributed according to workload. This ensures that the host and VMs receive sufficient memory.

Example:

- Total RAM = 16 GB
- Host = 4 GB
- Available = 12 GB
- VM1 = 2 GB
- VM2 = 2 GB
- VM3 = 2 GB
- VM4 = 2 GB
- VM5 = 2 GB
- Remaining = 2 GB

The remaining 2 GB can be kept as a buffer or dynamically assigned to VMs that require more memory.

CODE :

```
#include <stdio.h>
int main() {
  int totalRAM = 16;
```



```

int hostRAM = 4;
int availableRAM;
int minRAM = 2;
int n, i;
availableRAM = totalRAM - hostRAM;

printf("Total RAM: %d GB\n", totalRAM);
printf("Host Reserved RAM: %d GB\n", hostRAM);
printf("Enter number of VMs: ");
scanf("%d", &n);
printf("\nRAM Allocation:\n");
for (i = 1; i <= n; i++) {
    if (availableRAM >= minRAM) {
        printf("VM%d allocated %d GB\n", i, minRAM);
        availableRAM = availableRAM - minRAM;
    } else {
        printf("VM%d: Insufficient memory\n", i);
    }
}
printf("\nRemaining RAM: %d GB\n", availableRAM);
if (availableRAM >= 0)
    printf("RAM allocation completed successfully.\n");
return 0;
}

```

OUTPUT :

Output

```

Total RAM: 16 GB
Host Reserved RAM: 4 GB
Enter number of VMs: 3

RAM Allocation:
VM1 allocated 2 GB
VM2 allocated 2 GB
VM3 allocated 2 GB

Remaining RAM: 6 GB
RAM allocation completed successfully.

```

=== Code Execution Successful ===

4. Hypervisor Selection Algorithm :

Problem Analysis :

The choice between a Type-1 and Type-2 hypervisor depends on performance requirements, budget, and direct hardware access.

- Type-1: Runs directly on hardware and generally provides better performance.
- Type-2: Runs on top of a host operating system and is generally suitable for low-cost or desktop/testing environments.

Pseudocode :

```
BEGIN
  INPUT performance_required
  INPUT budget
  INPUT hardware_access
  IF performance_required = HIGH AND hardware_access = TRUE THEN
    selected_hypervisor = "Type-1 Hypervisor"
  ELSE IF budget = LOW AND hardware_access = FALSE THEN
    selected_hypervisor = "Type-2 Hypervisor"
  ELSE IF performance_required = HIGH THEN
    selected_hypervisor = "Type-1 Hypervisor"
  ELSE
    selected_hypervisor = "Type-2 Hypervisor"
  ENDIF
  PRINT "Selected Hypervisor:" PRINT selected_hypervisor
END
```

Explanation :

If high performance and direct hardware access are required, a Type-1 hypervisor is selected. If the system has a low budget and is mainly intended for testing or desktop use, a Type-2 hypervisor is suitable.

CODE :

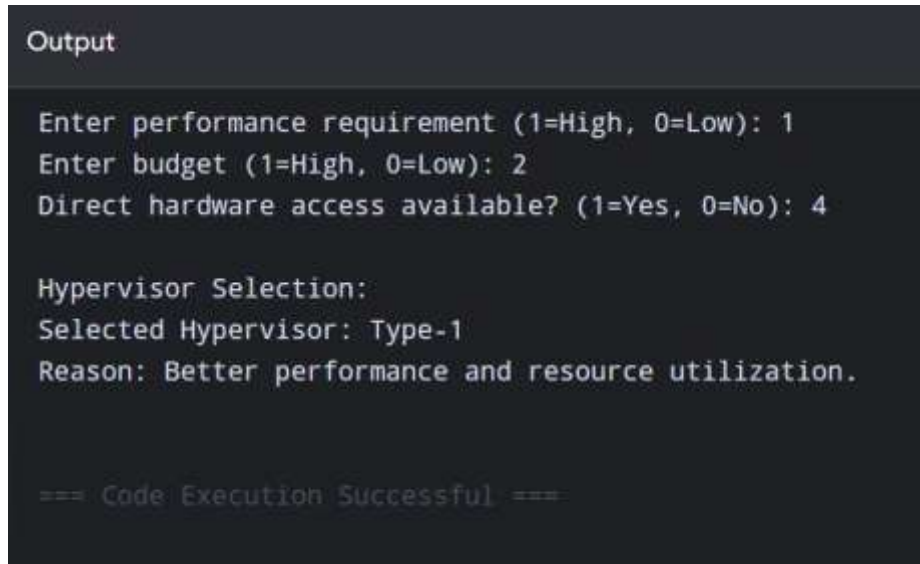
```
#include <stdio.h>
#include <string.h>
int main() {
  int performance;
  int budget;
  int hardware;
  printf("Enter performance requirement (1=High, 0=Low): ");
  scanf("%d", &performance);
  printf("Enter budget (1=High, 0=Low): ");
  scanf("%d", &budget);
  printf("Direct hardware access available? (1=Yes, 0=No): ");
  scanf("%d", &hardware);
  printf("\nHypervisor Selection:\n");
  if (performance == 1 && hardware == 1) {
    printf("Selected Hypervisor: Type-1\n");
    printf("Reason: High performance and direct hardware access.\n");
  }
}
```

```

else if (budget == 0) {
printf("Selected Hypervisor: Type-2\n");
printf("Reason: Low budget and suitable for general use.\n");
}
else {
printf("Selected Hypervisor: Type-1\n");
printf("Reason: Better performance and resource utilization.\n");
}
return 0;
}

```

OUTPUT :



```

Output

Enter performance requirement (1=High, 0=Low): 1
Enter budget (1=High, 0=Low): 2
Direct hardware access available? (1=Yes, 0=No): 4

Hypervisor Selection:
Selected Hypervisor: Type-1
Reason: Better performance and resource utilization.

=== Code Execution Successful ===

```

5. CPU and Memory Resource-Limiting Algorithm :

Problem Analysis:

Applications can consume excessive CPU and memory, affecting other applications. Resource control mechanisms such as Linux cgroups can be used to limit CPU and memory usage.

Pseudocode:

```

BEGIN
  INPUT application_list
  FOR each app IN application_list DO
    CREATE cgroup for app
    INPUT CPU_limit
    INPUT memory_limit
    SET CPU limit for app
    SET memory limit for app
    ASSIGN app process TO cgroup
    MONITOR CPU usage

    MONITOR memory usage
    IF CPU usage > CPU_limit THEN
      THROTTLE CPU usage
    ENDIF
    IF memory usage > memory_limit THEN

```

```

        LIMIT memory allocation
    ENDIF
ENDFOR
PRINT "Resource limits successfully applied"
END

```

Explanation :

A separate cgroup is created for each application. CPU and memory limits are assigned to prevent any single application from consuming excessive resources. The system monitors resource usage and throttles applications when they exceed their limits.

CODE :

```

#include <stdio.h>
struct Application {
char name[20];
int cpuLimit;
int memoryLimit;
int cpuUsage;
int memoryUsage;
};

int main() {
int n, i;
struct Application app[10];
printf("Enter number of applications: ");
scanf("%d", &n);
for (i = 0; i < n; i++) {
printf("\nEnter application name: ");

scanf("%s", app[i].name);
printf("Enter CPU limit (%%): ");
scanf("%d", &app[i].cpuLimit);
printf("Enter memory limit (MB): ");
scanf("%d", &app[i].memoryLimit);
printf("Enter current CPU usage (%%): ");
scanf("%d", &app[i].cpuUsage);
printf("Enter current memory usage (MB): ");
scanf("%d", &app[i].memoryUsage);
}
printf("\nResource Management Results\n");
printf("-----\n");
for (i = 0; i < n; i++) {
printf("\nApplication: %s\n", app[i].name);
if (app[i].cpuUsage > app[i].cpuLimit) {
printf("CPU limit exceeded -> Throttling CPU\n");
} else {
printf("CPU usage within limit\n");
}
if (app[i].memoryUsage > app[i].memoryLimit) {

```

```

printf("Memory limit exceeded -> Restricting memory\n");
} else {
printf("Memory usage within limit\n");
}
}
printf("\nResource limits successfully monitored.\n");
return 0;
}

```

OUTPUT :

Output

```

Enter number of applications: 2

Enter application name: Browser
Enter CPU limit (%): 40
Enter memory limit (MB): 2048
Enter current CPU usage (%): 60
Enter current memory usage (MB): 1800

Enter application name: Editor
Enter CPU limit (%): 30
Enter memory limit (MB): 1024
Enter current CPU usage (%): 20
Enter current memory usage (MB): 800

```

Output

```

Enter application name: Editor
Enter CPU limit (%): 30
Enter memory limit (MB): 1024
Enter current CPU usage (%): 20
Enter current memory usage (MB): 800

Resource Management Results
-----

Application: Browser
CPU limit exceeded -> Throttling CPU
Memory usage within limit

Application: Editor
CPU usage within limit
Memory usage within limit

```